

Lab 3 – Chat System

Objective: Design and implement a simple client/server chat system using Python and socket programming.

Details:

- **Architecture:** Develop a client-server architecture where multiple clients can connect to a central server.
- **Socket Programming:** Utilize Python's `socket` library to handle TCP connections.
- **Message Protocol:** Implement a simple text-based protocol for message exchange between clients and the server.
- **Concurrency:** Use threading or asynchronous programming to handle multiple client connections simultaneously.
- **Error Handling:** Implement robust error handling to manage connection issues, unexpected disconnections, and invalid messages.
- **User Interface:** Create a basic command-line interface (CLI) for both the client and server applications.
- **Logging:** Implement logging on the server side to keep track of client connections, disconnections, and message exchanges.

Deliverables:

- Source code for both client and server applications.
- Documentation explaining the design, implementation, and usage of the chat system.
- A brief report on the testing process and results.

// Documentation explaining the design, implementation, and usage of the chat system.

OVERVIEW

This is a relatively simple chat system that utilizes a client-server architecture to allow multiple clients to send messages to one another. This runs on a local machine with a single server that can handle concurrent messaging from multiple clients. Clients are assigned unique identifiers when they log onto the server for easier tracking and clearer identification in messaging.

This system takes a user input and broadcasts to all other users; direct messaging has not been implemented. Users are notified of who sent a message and when it is sent.

Below is an overview of the application organization.

```
chat_app/
├── common/
│   ├── __init__.py
│   ├── config.py           # Host, port, buffer size, encoding settings
│   ├── message_types.py    # Message constants (heartbeat, disconnect)
│   └── message_protocol.py  # ChatMessage class and JSON handling
├── client/
│   ├── __init__.py
│   ├── client.py           # Main client application
│   ├── chat_handler.py     # CLI interface and message display
│   ├── connection_manager.py # Manages server connection
│   ├── message_listener.py # Handles server messages
│   └── socket_manager.py   # Low-level socket operations
└── server/
    ├── __init__.py
    ├── server.py           # Main server application
    ├── socket_connection.py # Socket operations
    ├── message_handler.py  # Message processing
    ├── message_monitor.py  # Performance monitoring
    ├── connection_pool.py  # Manages multiple clients
    ├── connection_acceptor.py # Accepts new connections
    ├── client_connection.py # Individual client handler
    ├── client_lifecycle_manager.py # Connection lifecycle
    └── logger.py           # System-wide logging
```

Figure 1: System architecture. This does NOT contain the logs directory or files that do not contribute to main codebase.

The application is contained in a single package of 3 separate subsystems: common, client, and server.

- Common contains shared resources that can be utilized by client or server. This reduces redundancy of code and can increase cohesion with integration.
- Client is a module that contains the logic related to a single Client. *client.py* contains the main function for the module as well as general management of the Client.
- Server is a module that contains the logic related to the Server. *server.py* contains the main function for the module as well as general management of the Server.

Here is a brief description of the pipeline from client input to server broadcasting a message.

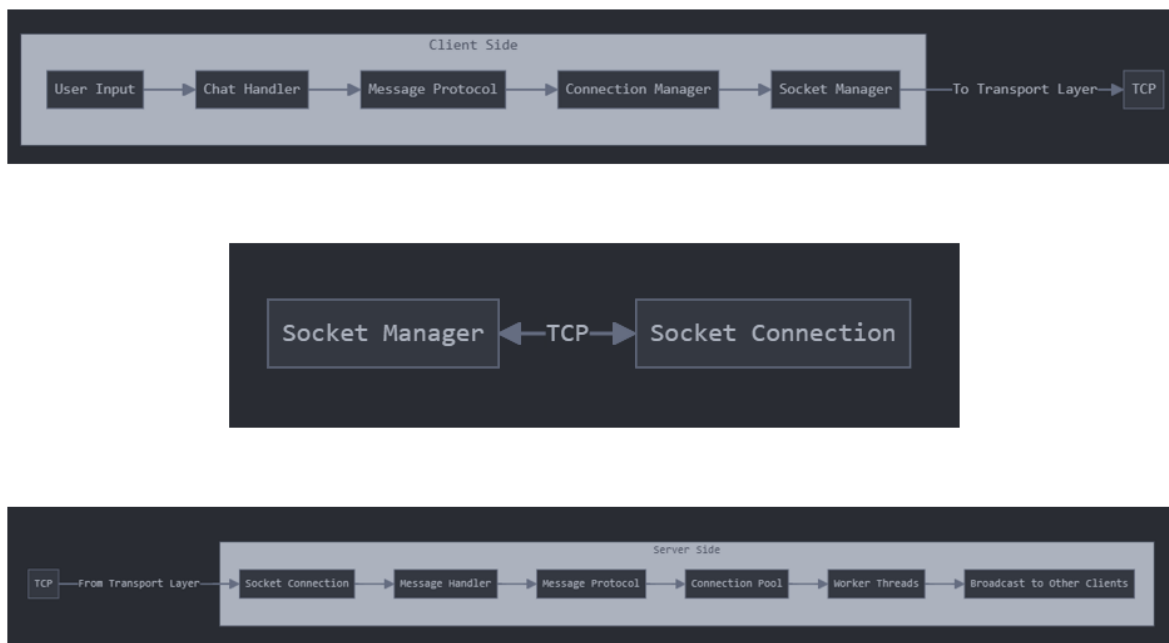


Figure 2 Client message flow via sockets to server broadcasting.

IMPLEMENTATION

This application is written using Python 3.12.9. It is designed to be modular, allowing an application to either run the client or the server. Client connections and TCP protocols utilize the *socket* library.

We utilize a local host. The server manages socket connections and message transmission. Only one instance of a server can be running at a time. If the server is not running, clients will be unable to connect and will terminate via timeout. While a Client is connected the Server sends periodic checks, designated “heartbeats” to test if the connection is open and the Client on that socket is responding.

The system heavily utilizes threading for concurrent operations. Each client connection runs in its own thread, allowing multiple clients to send and receive messages simultaneously. The server uses a worker

thread pool to handle message broadcasting, preventing any single slow client from impacting overall system performance. Thread safety is maintained through strategic use of locks. Message queues are implemented to handle the asynchronous nature of chat messages, with each worker thread processing messages from its dedicated queue to ensure orderly message distribution.

There are multiple layers managed by various classes to help control message flow properly.

1. Client

- a. The lowest level is the socket layer. This holds the primary logic to handle connecting and disconnecting, and direct sending and receiving of messages.
- b. The next layer is connection management. This layer is responsible for additional logic that goes with connecting and disconnecting. It also monitors the connection
- c. Next is message handling via *message_listener.py*. This part of the pipeline is responsible for controlling flow of messages based on message type.
- d. The final layer is the *chat_handler.py*. This is the direct user interface and holds the primary logic for the Command Line Interface.
- e. One last file is *client.py* which contains the Client's main function. This is a general manager coordinating the starting and stopping of the Client-side application.

2. Server

- a. The lowest level is the socket layer. *socket_connection.py* handles direct socket connections, sending and receiving raw data at the TCP level.
- b. The next layer is the connection layer which is split over multiple files:
 - i. *connection_acceptor.py* - handles incoming connection requests and creates new client instances
 - ii. *connection_pool.py* - manages all active connections and handles message broadcasting
 - iii. *client_connection.py* - manages individual client connections and their lifecycles
 - iv. *client_lifecycle_manager.py* - handles the state and behavior of each connected client
- c. Next is the message layer, again split over multiple files.
 - i. *message_handler.py* - processes incoming messages and manages message distribution
 - ii. *message_monitor.py* - tracks system performance and message statistics
- d. The logging layer handles system events, errors, and message tracking through file-based logging
- e. *server.py* holds CLI logic and management which contains the main server application logic and coordinates all component

USER GUIDE

Navigate to the root directory of the application. Open multiple terminals.

In terminal one, start the server.

```
t System\chat_system - final> python -m server.server
Connection pool initialized with 10 workers
Server is listening on 127.0.0.1:65432
Initial connection pool state: Active connections: 0 - Addresses: []
Server started successfully
```

In the other terminals, start clients. 3 will be opened and each will be assigned randomly generated but unique IDs:

```
\chat_system - final> python -m client.client
```

```
=== Chat Session Started ===
Logged in as: User7524
Type your message and press Enter to send
Type /online to see who's online
Press Ctrl+C to exit
=====
>

=== Chat Session Started ===
Logged in as: User9092
Type your message and press Enter to send
Type /online to see who's online
Press Ctrl+C to exit
=====
>

=== Chat Session Started ===
Logged in as: User6733
Type your message and press Enter to send
Type /online to see who's online
Press Ctrl+C to exit
=====
>
```

Figure 3 NOTE: the CLI will only show the chat while running.

We can look back at the Server CLI and see the connections.

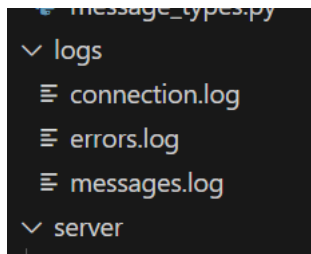
```
Server started successfully
New connection from ('127.0.0.1', 49169)
New connection from ('127.0.0.1', 49175)
New connection from ('127.0.0.1', 49177)
█
```

The Clients can freely send messages back and forth. Each message is broadcasted to the other Clients with who sent the message and a time stamp. The server also tracks this in a log file.

```
=== Chat Session Started ===
Logged in as: User7524
Type your message and press Enter to send
Type /online to see who's online
Press Ctrl+C to exit
=====
[20:49:42] You: Hello
[20:49:44] User9092: Hey
[20:49:48] User6733: Howdy
>

=== Chat Session Started ===
Logged in as: User9092
Type your message and press Enter to send
Type /online to see who's online
Press Ctrl+C to exit
=====
[20:49:42] User7524: Hello
[20:49:44] You: Hey
[20:49:48] User6733: Howdy
>

=== Chat Session Started ===
Logged in as: User6733
Type your message and press Enter to send
Type /online to see who's online
Press Ctrl+C to exit
=====
[20:49:42] User7524: Hello
[20:49:44] User9092: Hey
[20:49:48] You: Howdy
>
```



```
282 2025-02-22 20:49:40 [INFO] [('127.0.0.1', 49177)] SENT SUCCESS - HEARTBEAT_ACK
283 2025-02-22 20:49:42 [INFO] [('127.0.0.1', 49169)] BROADCAST SUCCESS - From User7524
284 2025-02-22 20:49:42 [INFO] [('127.0.0.1', 49169)] SENT SUCCESS - {"type": "CHAT", "user_id": "User7524", "content": "Hello", "timestamp": "20:49:42"}
285 2025-02-22 20:49:42 [INFO] [('127.0.0.1', 49175)] SENT SUCCESS - {"type": "CHAT", "user_id": "User7524", "content": "Hello", "timestamp": "20:49:42"}
286 2025-02-22 20:49:42 [INFO] [('127.0.0.1', 49177)] SENT SUCCESS - {"type": "CHAT", "user_id": "User7524", "content": "Hello", "timestamp": "20:49:42"}
287 2025-02-22 20:49:42 [INFO] [('127.0.0.1', 49175)] SENT SUCCESS - HEARTBEAT_ACK
288 2025-02-22 20:49:42 [INFO] [('127.0.0.1', 49169)] SENT SUCCESS - HEARTBEAT_ACK
289 2025-02-22 20:49:44 [INFO] [('127.0.0.1', 49175)] BROADCAST SUCCESS - From User9092
290 2025-02-22 20:49:44 [INFO] [('127.0.0.1', 49175)] SENT SUCCESS - {"type": "CHAT", "user_id": "User9092", "content": "Hey", "timestamp": "20:49:44"}
291 2025-02-22 20:49:44 [INFO] [('127.0.0.1', 49169)] SENT SUCCESS - {"type": "CHAT", "user_id": "User9092", "content": "Hey", "timestamp": "20:49:44"}
292 2025-02-22 20:49:44 [INFO] [('127.0.0.1', 49177)] SENT SUCCESS - {"type": "CHAT", "user_id": "User9092", "content": "Hey", "timestamp": "20:49:44"}
293 2025-02-22 20:49:45 [INFO] [('127.0.0.1', 49177)] SENT SUCCESS - HEARTBEAT_ACK
294 2025-02-22 20:49:47 [INFO] [('127.0.0.1', 49175)] SENT SUCCESS - HEARTBEAT_ACK
295 2025-02-22 20:49:47 [INFO] [('127.0.0.1', 49169)] SENT SUCCESS - HEARTBEAT_ACK
296 2025-02-22 20:49:48 [INFO] [('127.0.0.1', 49177)] BROADCAST SUCCESS - From User6733
297 2025-02-22 20:49:48 [INFO] [('127.0.0.1', 49169)] SENT SUCCESS - {"type": "CHAT", "user_id": "User6733", "content": "Howdy", "timestamp": "20:49:48"}
298 2025-02-22 20:49:48 [INFO] [('127.0.0.1', 49175)] SENT SUCCESS - {"type": "CHAT", "user_id": "User6733", "content": "Howdy", "timestamp": "20:49:48"}
299 2025-02-22 20:49:48 [INFO] [('127.0.0.1', 49177)] SENT SUCCESS - {"type": "CHAT", "user_id": "User6733", "content": "Howdy", "timestamp": "20:49:48"}
300 2025-02-22 20:49:50 [INFO] [('127.0.0.1', 49177)] SENT SUCCESS - HEARTBEAT_ACK
```

The Client will shutdown one of two ways

1) The Client may choose to logoff on their own with Ctrl+C

```
Press Ctrl+C to exit
=====
[20:49:42] You: Hello
[20:49:44] User9092: Hey
[20:49:48] User6733: Howdy
[20:52:36] You: Time to sign off
>
Received interrupt signal, initiating shut
down...

Stopping chat handler...
Chat handler stopped
Initiating disconnect...
Message listener loop ended
Message listener stopped
Socket cleanup completed
Disconnect completed
```

2) The Server needs to shut down (Ctrl+C) and broadcasts a signal to have the Clients shut down.

```
Connection pool initialized with 10 workers
Server is listening on 127.0.0.1:65432
Initial connection pool state: Active connections: 0 - Addresses: []
Server started successfully
New connection from ('127.0.0.1', 49169)
New connection from ('127.0.0.1', 49175)
New connection from ('127.0.0.1', 49177)

Received interrupt signal, initiating shutdown...
Shutting down server...
Stopping connection acceptor...
Starting connection pool cleanup...
Connection pool cleanup complete
Server shutdown complete
```

```
Press Ctrl+C to exit
=====
```

```
[20:49:42] User7524: Hello
[20:49:44] You: Hey
[20:49:48] User6733: Howdy
[20:52:36] User7524: Time to sign off
> Received server shutdown message
Message listener loop ended
```

```
Stopping chat handler...
Chat handler stopped
Initiating disconnect...
Message listener stopped
Socket cleanup completed
Disconnect completed
```

```
Press Ctrl+C to exit
=====
```

```
[20:49:42] User7524: Hello
[20:49:44] User9092: Hey
[20:49:48] You: Howdy
[20:52:36] User7524: Time to sign off
> Received server shutdown message
Message listener loop ended
```

```
Stopping chat handler...
Chat handler stopped
Initiating disconnect...
Message listener stopped
Socket cleanup completed
Disconnect completed
```

The server logs these events in a file:

```
83 2025-02-22 20:43:53 [INFO] [('127.0.0.1', 49169)] Starting client lifecycle
84 2025-02-22 20:45:55 [INFO] [('127.0.0.1', 49175)] Starting client lifecycle
85 2025-02-22 20:45:57 [INFO] [('127.0.0.1', 49177)] Starting client lifecycle
86 2025-02-22 20:49:42 [INFO] [('127.0.0.1', 49169)] User ID updated to User7524
87 2025-02-22 20:49:44 [INFO] [('127.0.0.1', 49175)] User ID updated to User9092
88 2025-02-22 20:49:48 [INFO] [('127.0.0.1', 49177)] User ID updated to User6733
89 2025-02-22 20:52:44 [INFO] [('127.0.0.1', 49169)] User User7524 disconnected
90 2025-02-22 20:52:44 [INFO] [('127.0.0.1', 49169)] Starting client cleanup for User User7524
91 2025-02-22 20:52:44 [INFO] [('127.0.0.1', 49169)] Client cleanup completed for User User7524
92 2025-02-22 20:52:48 [INFO] [('127.0.0.1', 49175)] Sending shutdown notification
93 2025-02-22 20:52:48 [INFO] [('127.0.0.1', 49175)] Shutdown notification sent
94 2025-02-22 20:52:48 [INFO] [('127.0.0.1', 49175)] User User9092 acknowledged shutdown
95 2025-02-22 20:52:48 [INFO] [('127.0.0.1', 49175)] Starting client cleanup for User User9092
96 2025-02-22 20:52:48 [INFO] [('127.0.0.1', 49175)] Client cleanup completed for User User9092
97 2025-02-22 20:52:48 [INFO] [('127.0.0.1', 49177)] Sending shutdown notification
98 2025-02-22 20:52:48 [INFO] [('127.0.0.1', 49177)] Shutdown notification sent
99 2025-02-22 20:52:48 [INFO] [('127.0.0.1', 49177)] Starting client cleanup for User User6733
100 2025-02-22 20:52:48 [INFO] [('127.0.0.1', 49177)] User User6733 acknowledged shutdown
101 2025-02-22 20:52:48 [INFO] [('127.0.0.1', 49177)] Client cleanup completed for User User6733
102
```

The server logs Errors to a file if they arise:

```
1 2025-02-22 18:42:00 [ERROR] [('127.0.0.1', 64631)] ONLINE_USERS_REQUEST - Failed to parse message
2
```

// Testing and results

System testing and integration testing was implemented throughout the development process. Tests were conducted manually through running the application. Testing for client and server behaviors are more involved than standalone applications due to the coordination involved.

Connections:

During development I played with the number of clients connecting to the server. Initially, I worked through a single client being able to connect. Once that was established, then a second was connected, then a third. After 3, I wanted to stress the system and connected 10 clients which the system allowed. I did not proceed more due to screen space and time.

Shutdown Protocols:

The shutdown sequences were particularly problematic with race conditions. It is easier to handle for a single client, but coordination with the server has caused issues. What I would do is use the signal interrupt Ctrl+C on a Client to ensure that it was working as intended. One key component is a message the Client sends to the Server to let the Server know that it is closing. I created a log for this so that I could verify. Then, I would attempt Server closing of the Clients and traced each step with logs to identify stuck points or logical flaws.

Latency Issues and Message Protocols:

At one point there was extreme lag occurring when messages were being transmitted. Some messages were quick and others would take a few seconds. As this became more frequent, I would spam the system from single client then multiple clients, trying to overwhelm this system in an attempt to expose a flaw. I was able to identify 2 major issues: One, the message broadcasting was happening sequentially and not concurrently and two, locks were not releasing causing deadlocking. Addressing these made messaging appear instantaneous even with 10 clients.

UNRESOLVED ISSUES

Issue #1:

There were instances when the Client would type and a longer message would duplicated repeatedly with every additional letter typed. It has been identified to be an issue when the width of a message exceeds a single line.

Issue #2:

A command system was intended to be implemented which would begin with a /, such as “/online” to show Users online. This was not fully implemented; therefore, the Client can transmit these with some showing errors to the server. It is not explained to the user, but /help brings up commands.

```
=== Available Commands ===  
/online - Show online users  
/clear  - Clear screen  
/help   - Show this help  
=====  
> |
```

CONCLUSION

This project was relatively involved and tested my skills. I found myself becoming excited and in turn, overly ambitious. I began attempting to work in features that would improve UX but caused delays or ended up breaking some of the code resulting in lost time.

The client-server architecture and handling resource management and multiuser considerations was different enough of a problem that the challenges took quite a bit of thought. I found myself refactoring and reworking protocols, methods, and subsystems so many times that the overall structure is not was originally envisioned and the application suffers in some ways, such as hindered scalability. This project would have been easier and more manageable if I had taken some extra time reviewing requirements and thinking through the business logic. On the flip side, working through the project exposed what the business logic was and will help me in future projects.

One of the deficits in this application is the lack of interfaces. Logic is more intertwined and couple than I was planning, but as features were added and incorporated, I found I was placing methods in places that made sense at the time. Once again, a step back with planning would have made this process more efficient with a more modular and scalable code.

Exploring sockets and the various issues that can arise with multi-user systems, such as collisions and race conditions, were really apparent. Working through building this application has started to train me on signs and symptoms of when threading or concurrency issues started to take effect, similar to how a mechanic may know what part to look at on a vehicle based on the clunky sound it makes.

Ultimately, I am pleased with the result. I chose to go with a broadcasting system. That drove a lot of considerations for how Client chat messages were to be handled. The system runs fairly efficiently and appears to have no delays from Client transmission to displaying on the other clients CLI.